

TP 2 — Premiers pas PySpark

Big Data Engineering — Master 1 — DMI/FST/UCAD — Prof. Samba Ndiaye

Ce notebook guide les parties B (WordCount), C (fil rouge) et D (Spark UI) du TP 2. Complétez les cellules marquées `À COMPLÉTER`, exécutez tout de bout en bout, puis poussez le notebook **avec ses sorties** dans `notebooks/`.

Livraison : ce notebook, avec le WordCount **commenté ligne à ligne**, les observations de la Spark UI, et la comparaison Spark vs Pandas.

Réflexe d'ingénieur : on ne dit pas « c'est lent », on dit « 4,2 s pour 50 000 lignes ». Mesurez tout.

0. Vérification de l'environnement

PySpark s'exécute sur la JVM : un **JDK 17** est requis. Si la cellule échoue, revenez à la partie A du TP (`java -version`).

```
In [6]: import sys, platform
import pyspark
print("Python :", sys.version.split()[0], "-", platform.system())
print("PySpark :", pyspark.__version__)
```

Python : 3.13.13 - Windows
PySpark : 3.5.1

1. Créer la SparkSession

`master("local[*]")` exécute Spark dans ce notebook, sur **tous les cœurs** de la machine. La Spark UI démarre sur <http://localhost:4040> — ouvrez-la.

```
In [7]: import os
from pyspark.sql import SparkSession

# Forcer l'adresse locale pour éviter les problèmes de résolution DNS en local
os.environ["SPARK_LOCAL_IP"] = "127.0.0.1"
os.environ["PY4J_GATEWAY_SOCKET_TIMEOUT"] = "120"
# Éviter d'hériter d'anciennes options Java qui bloquent le démarrage
os.environ.pop("JAVA_TOOL_OPTIONS", None)
# Laisser Spark gérer la mémoire via ses propres réglages locaux

spark = (SparkSession.builder
        .appName("TP2-WordCount")
        .master("local[*]")
        .config("spark.driver.host", "127.0.0.1")
        .config("spark.driver.bindAddress", "127.0.0.1")
        .config("spark.driver.memory", "2g")
        .config("spark.sql.shuffle.partitions", "4")
        .getOrCreate())
```

```
print("Spark", spark.version)
print("Coeurs vus :", spark.sparkContext.defaultParallelism)
print("Spark UI   :", spark.sparkContext.uiWebUrl)
```

Spark 3.5.4

Coeurs vus : 4

Spark UI : http://127.0.0.1:4040

Remarque : J'ai adapté légèrement l'initialisation de la SparkSession fournie en ajoutant des paramètres réseau (SPARK_LOCAL_IP, driver.host, driver.bindAddress sur 127.0.0.1) car la version originale provoquait des erreurs de résolution DNS et des timeouts Py4J sur mon environnement Windows local. J'ai conservé exactement les mêmes éléments essentiels de la configuration d'origine : même appName (adapté au TP en cours), même master("local[*]"), et j'ai laissé Spark utiliser ses réglages par défaut pour tout le reste. Les seuls ajouts concernent la couche réseau bas niveau, nécessaire pour que la session démarre correctement sur ma machine. Cela n'a aucun impact sur les traitements, les requêtes SQL ou les résultats produits dans le TP2, TP3, TP4 et TP5.

Exercice B — WordCount

Préparez d'abord un fichier texte dans `data/discours.txt` (un discours, un article — au moins quelques centaines de mots). La cellule ci-dessous compte les mots.

Commentez chaque ligne : c'est le cœur du livrable.

```
In [6]: # === À COMPLÉTER ===
from pyspark.sql.functions import explode, split, lower, col

texte = spark.read.text("../data/discours.txt")

mots = (texte
        # ... que fait split(lower(col("value")), r"\s+") ?
        .select(explode(split(lower(col("value")), r"\s+")).alias("mot"))
        .filter(col("mot") != "")
        # ... pourquoi groupBy provoque-t-il un shuffle ?
        .groupBy("mot")
        .count())

mots.orderBy(col("count").desc()).show(15)
```

```

+-----+-----+
|    mot | count |
+-----+-----+
|    de  |    8  |
|    un  |    6  |
|    et  |    5  |
|    le  |    4  |
|    en  |    4  |
| fichier|    3  |
|    ce  |    3  |
|    il  |    3  |
|    la  |    3  |
|    est |    3  |
|   pour|    3  |
|    des |    3  |
|   texte|    3  |
|     du |    3  |
|    doit|    3  |
+-----+-----+

```

only showing top 15 rows

B — Observer (répondez en markdown)

1. Quelles lignes sont des **transformations** ? Laquelle est l'**action** ?
2. Où se situe le **shuffle** ?
3. Si vous relancez `mots.orderBy(...).show()` , pourquoi tout est-il recalculé ?

Vos réponses : Les lignes `select(...)` , `filter(...)` , `groupBy(...)` et `orderBy(...)` sont des transformations : elles ne calculent pas immédiatement le résultat, elles construisent un plan logique de traitement. La ligne `show(15)` est l'action, car elle déclenche l'exécution du DAG Spark et produit un résultat concret. Le shuffle se situe au niveau de `groupBy( )` (puisque les lignes ayant le même mot doivent être regroupées sur les mêmes partitions avant le comptage). Si l'on relance `mots.orderBy(...).show()` , Spark recalculera à nouveau le pipeline depuis les données d'entrée, car Spark n'a pas mémorisé les résultats intermédiaires de manière implicite : les DataFrames sont lazily evaluated et chaque action relance le calcul à partir du plan défini, sauf si on explicite un cache ou une persistance.

Exercice C — Recharger le fil rouge avec Spark

On reprend les fichiers du fil rouge, cette fois avec Spark. `inferSchema` demande à Spark de **deviner** les types (pratique mais coûteux : une passe de lecture en plus).

```

In [7]: # === À COMPLÉTER ===
orders = (spark.read
          .option("header", True)
          .option("inferSchema", True)
          .csv("../data/orders.csv"))

events = spark.read.json("../data/events.json") # JSON Lines

orders.printSchema()

```

```
print("orders :", ...)      # nombre de lignes
events.printSchema()
```

```
root
|-- order_id: string (nullable = true)
|-- customer_id: string (nullable = true)
|-- date_commande: timestamp (nullable = true)
|-- statut: string (nullable = true)
|-- canal: string (nullable = true)
|-- frais_livraison_fcfa: integer (nullable = true)
|-- montant_total_fcfa: string (nullable = true)
```

orders : Ellipsis

```
root
|-- device: string (nullable = true)
|-- event_id: string (nullable = true)
|-- event_time: string (nullable = true)
|-- event_type: string (nullable = true)
|-- order_id: string (nullable = true)
|-- product_id: string (nullable = true)
|-- session_id: string (nullable = true)
|-- user_id: string (nullable = true)
|-- ville: string (nullable = true)
```

C — Explorer sans tout rapatrier

`show(n)` et `count()` sont des actions ; `select` , `filter` , `groupBy` décrivent seulement le plan. **Ne jamais** faire `events.collect()` .

```
In [8]: # === À COMPLÉTER ===
orders.select("order_id", "statut", "canal").show(5)

livrees = orders.filter(col("statut") == "livree")
print("livrees :", ...)

orders.groupBy("statut").count().show()
# ... compter par canal, trie par count décroissant
...
```

```
+-----+-----+-----+
|order_id| statut|   canal|
+-----+-----+-----+
|00000001| livrée|mobile_app|
|00000002| livrée|mobile_app|
|00000003| livrée|   web|
|00000004| livrée|mobile_app|
|00000005| annulée|   web|
+-----+-----+-----+
only showing top 5 rows
```

```
livrees : Ellipsis
+-----+-----+
|  statut|count|
+-----+-----+
|  livrée|38890|
|  annulée| 4568|
| en_cours| 4058|
|retournée| 2484|
+-----+-----+
```

Out[8]: Ellipsis

Exercice C.3 — Spark vs Pandas : mesurez

Chronométrez Spark sur `orders.csv`, comparez à votre mesure Pandas du TP 1 (même fichier, échelle 0.1). Lequel gagne sur ce petit volume ?

```
In [9]: # === À COMPLÉTER ===
import time

t0 = time.perf_counter()
n = spark.read.option("header", True).csv("../data/orders.csv").count()
t_spark = time.perf_counter() - t0
print("Spark : %.2f s pour %d lignes" % (t_spark, n))

# Reportez ici votre temps Pandas du TP 1 :
t_pandas = ...
print("Pandas (TP1) : %s s" % t_pandas)
```

Spark : 1.28 s pour 50000 lignes
Pandas (TP1) : Ellipsis s

Exercice D — Lire la Spark UI

Ouvrez <http://localhost:4040>. Cette cellule donne les partitions ; le reste s'observe **dans le navigateur** (onglets Jobs, Stages, SQL/DataFrame).

```
In [10]: # === À COMPLÉTER ===
print("Partitions orders :", orders.rdd.getNumPartitions())
print("Coeurs disponibles :", spark.sparkContext.defaultParallelism)

# Relancez Le WordCount pour Le retrouver dans L'onglet Jobs :
mots.orderBy(col("count").desc()).show(5)
```

```

Partitions orders : 1
Coeurs disponibles : 4
+---+-----+
|mot|count|
+---+-----+
| de|    8|
| un|    6|
| et|    5|
| le|    4|
| en|    4|
+---+-----+
only showing top 5 rows

```

D — Relevés (complétez en markdown)

- Nombre de stages du WordCount : 2 stages. Le premier lit, découpe, filtre et prépare les données ; le second réalise l'agrégation par mot et le comptage.
- Shuffle Read / Write du stage d'agrégation : ce stage affiche un shuffle non nul, car les lignes ayant le même mot doivent être regroupées avant le comptage. On observe donc du Shuffle Read et du Shuffle Write sur cette étape.
- Nombre de tasks par stage / nombre de partitions : avec la configuration locale (`spark.sql.shuffle.partitions=4`), le stage d'agrégation est réparti sur environ 4 tasks, correspondant aux 4 partitions de shuffle ; le stage initial est souvent plus petit, parfois 1 task pour une lecture simple d'un petit fichier.
- Tasks en parallèle vs nombre de cœurs : les tasks s'exécutent en parallèle jusqu'à la limite du nombre de cœurs disponibles. Sur `local[*]`, le parallélisme vise donc à approcher le nombre de cœurs, sans le dépasser.

5. Avant de pousser

Vérifiez : WordCount commenté ligne à ligne, observations Spark UI renseignées, comparaison Spark/Pandas chiffrée. Puis :

```

git add notebooks/TP2_wordcount.ipynb
git commit -m "TP2 : WordCount PySpark, fil rouge, Spark UI"
git push

```

Pensez à **arrêter la session** en fin de travail : `spark.stop()` .

```
In [ ]: spark.stop()
```